Code: Stretch Sensor Breathing Detection (Arduino)

Project reference: STN22_Thoracic belt for respiratory rate measurement
Authors: Hakkou Dounia
Date: 2025–2026 © SIS TEAM NANCY

File header

```
/*
 * Description: Respiratory rate measurement firmware for Heltec WiFi LoRa 32 V3.
 * Acquires signal from a stretch sensor, applies a 3-stage filtering chain
 * (median filter, low-pass filter, moving average), detects respiratory cycles
 * via zero-crossing with hysteresis, computes RPM over a 1-minute sliding window,
 * and transmits the result via LoRa every 5 seconds.
 *
 * Project reference: STN22_Thoracic belt FR
 * Authors:Hakkou Dounia
 * Date: 2025-2026
 * © SIS TEAM NANCY
 */
```

Libraries

```
// Arduino core framework
#include <Arduino.h>
// I2C communication (required by the OLED driver)
#include <Wire.h>
// OLED display driver for SSD1306 — ThingPulse library
#include "SSD1306Wire.h"
// Heltec V3 board-specific pin definitions (Vext, RST_OLED, SDA_OLED, SCL_OLED...)
#include "pins_arduino.h"
// Universal radio library for LoRa SX1262 (jgromes/RadioLib)
#include <RadioLib.h>
```

OLED display

```
// Instantiate the OLED on I2C address 0x3C using Heltec V3 SDA/SCL pins
SSD1306Wire display(0x3c, SDA_OLED, SCL_OLED);

// Enable the external power rail (Vext) required to power the OLED on Heltec V3
void VextON() {
  pinMode(Vext, OUTPUT);
  digitalWrite(Vext, LOW);   // LOW = Vext ON on Heltec V3
}

// Hardware reset sequence for the OLED controller (HIGH → LOW → HIGH)
void displayReset() {
  pinMode(RST_OLED, OUTPUT);
  digitalWrite(RST_OLED, HIGH); delay(20);
  digitalWrite(RST_OLED, LOW);  delay(20);
  digitalWrite(RST_OLED, HIGH); delay(20);
}
```

LoRa module (SX1262)

```
// SPI chip-select and control pins for the SX1262 LoRa transceiver on Heltec V3
#define LORA_SS    8    // SPI chip select
#define LORA_DIO1  14   // Interrupt / busy line
#define LORA_RST   12   // Hardware reset
#define LORA_BUSY  13   // Busy signal

// Instantiate the RadioLib module and SX1262 radio object
Module loraModule(LORA_SS, LORA_DIO1, LORA_RST, LORA_BUSY);
```

```
SX1262 radio(&loraModule);
```

Sensor & filter variables

```
// Analog input pin connected to the stretch sensor voltage divider (10 kΩ + sensor)
const int stretchPin = 7;

// Median filter
// Circular buffer of 5 samples; suppresses impulse noise (movement artefacts)
const int MEDIAN_SIZE = 5;
float medianBuffer[MEDIAN_SIZE] = {0};
int   medianIndex = 0;

// Exponential low-pass filter
// Smoothing coefficient alpha = 0.5  (0 = max smoothing, 1 = no smoothing)
// Formula: filteredValue = alpha x median + (1-alpha) x filteredValue
float filteredValue = 0;
const float alpha = 0.5;

// Moving average (dynamic baseline)
// 150-sample circular buffer (~4.5 s at 33 Hz); tracks slow baseline drift
const int AVG_SIZE = 150;
float avgBuffer[AVG_SIZE] = {0};
int   avgIndex = 0;
float avgSum   = 0;
float movingAvg = 0;
```

Breath detection variables

```
// Zero-crossing counter: incremented on each threshold crossing (insp. or exp.)
int zeroCrossings = 0;
// Total breath cycles counted within the current 1-minute window
int breathCount = 0;
// Timestamp of the last validated breath cycle (ms since boot)
unsigned long lastBreathTime = 0;
// Minimum interval between two consecutive breath cycles (ms)
// Dynamically updated at each window reset to match measured breathing rate
unsigned long MIN_BREATH_INTERVAL = 1200;
// Hysteresis thresholds (+/-8 ADC units on the centred signal)
// Inspiration: centredValue > +HYSTERESIS_THRESHOLD
// Expiration: centredValue < -HYSTERESIS_THRESHOLD
const float HYSTERESIS_THRESHOLD = 8;
// State flags preventing double-counting within a single phase
bool lastWasAbove = false;
bool lastWasBelow = false;
```

Calibration variables

```
// Flag set to true once the initial baseline has been computed
bool isCalibrated = false;
// Number of raw samples collected during the warm-up calibration phase
const int CALIBRATION_SAMPLES = 150;
int   calibrationCount = 0;
float baselineSum = 0;
```

Timing variables

```
// RPM sliding window: 1 minute (60 000 ms)
unsigned long windowStart   = 0;
const unsigned long WINDOW_DURATION = 60000;
// Main loop period: 30 ms -> sampling rate ~33 Hz
unsigned long lastLoopTime  = 0;
const unsigned long LOOP_INTERVAL = 30;
```

```
// LoRa transmission interval: every 5 seconds
unsigned long lastLoraSend  = 0;
const unsigned long LORA_SEND_INTERVAL = 5000;
```

Function: median filter

```
// Returns the median value of a float array using an in-place bubble sort.
// A local copy is sorted so the original circular buffer is not modified.
float getMedian(float* array, int size) {
  float sorted[size];
  memcpy(sorted, array, size * sizeof(float));
  // Bubble sort: O(n^2),acceptable for n=5
  for (int i = 0; i < size - 1; i++) {
    for (int j = i + 1; j < size; j++) {
      if (sorted[i] > sorted[j]) {
        float t = sorted[i]; sorted[i] = sorted[j]; sorted[j] = t;
      }
    }
  }
  // Return the middle element (index size/2 = 2 for size=5)
  return sorted[size / 2];
}
```

Setup

```
void setup() {
  Serial.begin(115200);
  delay(1000);          // Allow USB-serial to stabilise

  // Power on and reset the OLED, then initialise and flip for correct orientation
  VextON();
  delay(100);
  displayReset();
  display.init();
  display.flipScreenVertically();
  display.setFont(ArialMT_Plain_16);
  display.clear();
  display.drawString(0, 20, "Etalonnage...");
  display.display();

  // Set ADC resolution to 12 bits (0-4095) for finer signal granularity
  analogReadResolution(12);

  // Initialise LoRa SX1262:
  // freq=868.0 MHz (EU ISM), BW=125 kHz, SF=7, CR=5,
  // syncWord=0x12, txPower=14 dBm (EU legal limit), preamble=8
  int state = radio.begin(868.0, 125.0, 7, 5, 0x12, 14, 8);
  if (state != RADIOLIB_ERR_NONE) {
    Serial.print("LoRa init error: ");
    Serial.println(state);
    while (1);   // Halt LoRa is critical for data transmission
  }
  windowStart = millis();
}
```

Main loop

```
void loop() {
  unsigned long currentTime = millis();

  // Enforce 30 ms sampling period (non-blocking delay)
  if (currentTime - lastLoopTime < LOOP_INTERVAL) return;
  lastLoopTime = currentTime;

  // Read raw ADC value from the stretch sensor voltage divider
```

```
float rawValue = (float)analogRead(stretchPin);

// CALIBRATION PHASE
// Accumulate raw samples to compute an initial baseline before detection starts
if (!isCalibrated) {
  baselineSum += rawValue;
  calibrationCount++;
  if (calibrationCount >= CALIBRATION_SAMPLES) {
    // Average of 150 samples -> initial baseline and moving-average seed
    filteredValue = baselineSum / CALIBRATION_SAMPLES;
    movingAvg     = filteredValue;
    isCalibrated  = true;
    Serial.println("Etalonnage_termine:1");
  }
  return;  // Skip detection until calibration is complete
}

// MEDIAN FILTER
// Insert new sample into the circular buffer and compute the median
medianBuffer[medianIndex] = rawValue;
medianIndex = (medianIndex + 1) % MEDIAN_SIZE;
float medianValue = getMedian(medianBuffer, MEDIAN_SIZE);

// EXPONENTIAL LOW-PASS FILTER
// Attenuates high-frequency noise remaining after the median filter
filteredValue = alpha * medianValue + (1 - alpha) * filteredValue;

// MOVING AVERAGE (dynamic baseline)
// Update the circular sum: subtract oldest value, add new filtered value
avgSum -= avgBuffer[avgIndex];
avgBuffer[avgIndex] = filteredValue;
avgSum += filteredValue;
avgIndex = (avgIndex + 1) % AVG_SIZE;
movingAvg = avgSum / AVG_SIZE;

// Centre the signal around zero by subtracting the dynamic baseline
float centeredValue = filteredValue - movingAvg;

// Uncomment to visualise signal and thresholds in Arduino Serial Plotter:
// Serial.print("Signal:"); Serial.print(centeredValue);
// Serial.print(",Seuil_Haut:"); Serial.print(HYSTERESIS_THRESHOLD);
// Serial.print(",Seuil_Bas:"); Serial.println(-HYSTERESIS_THRESHOLD);

// BREATH DETECTION: ZERO-CROSSING WITH HYSTERESIS
// Inspiration validated when centred signal exceeds the upper threshold
if (!lastWasAbove && centeredValue > HYSTERESIS_THRESHOLD) {
  lastWasAbove = true;
  lastWasBelow = false;
  zeroCrossings++;
  Serial.println("Action:INSPIRATION");
}
// Expiration validated when centred signal falls below the lower threshold
else if (!lastWasBelow && centeredValue < -HYSTERESIS_THRESHOLD) {
  lastWasBelow = true;
  lastWasAbove = false;
  zeroCrossings++;
  Serial.println("Action:EXPIRATION");
}

// One complete breath = 2 threshold crossings (inspiration + expiration)
// Minimum interval guard prevents double-counting at high breathing rates
if (zeroCrossings >= 2) {
  if ((currentTime - lastBreathTime) > MIN_BREATH_INTERVAL) {
    breathCount++;
    lastBreathTime = currentTime;
    Serial.print("Cycle_Complet:");
    Serial.println(breathCount);
  }
  zeroCrossings = 0;
```

```
  }

  // RPM CALCULATION
  // Instantaneous RPM over the elapsed portion of the 1-minute window
  float bpm = (breathCount * 60000.0) / max(1UL, (currentTime - windowStart));

  // LORA TRANSMISSION
  // Send RPM every 5 s as a compact ASCII packet: "R:<value>"
  if (currentTime - lastLoraSend >= LORA_SEND_INTERVAL) {
    lastLoraSend = currentTime;
    String packet = "R:" + String(bpm, 1);
    radio.transmit(packet);
  }

  // OLED DISPLAY
  // Refresh screen each loop cycle with current RPM and cycle count
  display.clear();
  display.drawString(0,  0, "Freq Respi");
  display.drawString(0, 20, "RPM: " + String(bpm, 1));
  display.drawString(0, 40, "Cycles: " + String(breathCount));
  display.display();

  // WINDOW RESET
  // At the end of each 1-minute window, reset counters and recalculate
  // MIN_BREATH_INTERVAL based on measured RPM to handle high breathing rates
  if (currentTime - windowStart >= WINDOW_DURATION) {
    breathCount    = 0;
    zeroCrossings  = 0;
    lastWasAbove   = false;
    lastWasBelow   = false;
    windowStart    = currentTime;
    // Dynamic minimum interval: 18000/RPM gives the half-period in ms
    // Example: 30 RPM -> 600 ms | 60 RPM -> 300 ms
    if (bpm > 0) MIN_BREATH_INTERVAL = 18000 / bpm;
  }
}
```